

SOME KEY ATTRIBUTES OF ISAs

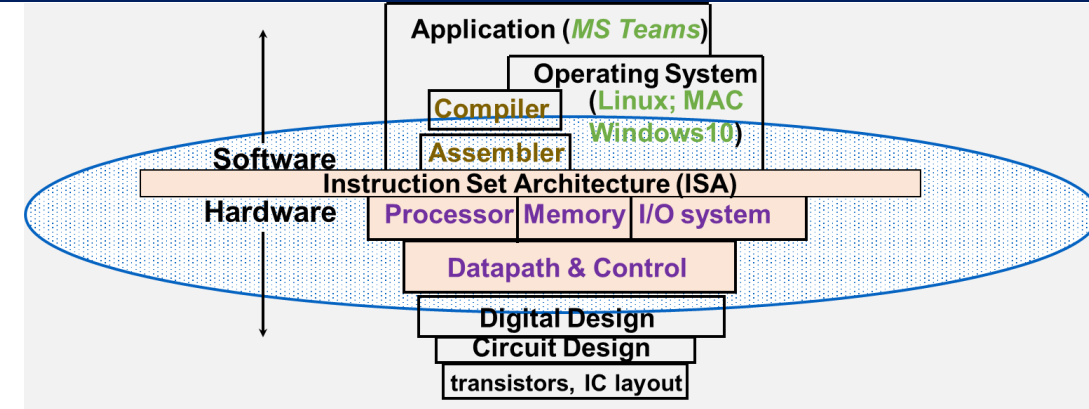
- Instruction encoding
 - Fixed length (32-bit or 64-bit for MIPS, ARM)
 - Variable length (1 byte to 16 bytes, average of ~3 bytes, x86_64)
- Number and type of registers
 - MIPS has 32 “integer” registers and 32 “floating point” registers
 - ARM & x86 both have 16 “integer” registers and 16 “floating point” registers
- Address space
 - ARM: 32-bit addresses at 8-bit granularly (2^{32} or 4GB total)
 - Modern x86 and ARM64: 64-bit addresses (2^{32} or 16 ExaB!)
- Memory addressing modes
 - MIPS: address calculated by “register+offset”
 - x86 and others have much more complicated addressing modes.

- Complex instruction set computing (CISC)
 - Many instructions (several hundreds)
 - An instruction takes many cycles to execute
 - Examples: _x86(Intel Pentium), VAX (vax11), PDP(pdp8)
- Reduced instruction set computing (RISC)
 - Small set of instructions (typically 32)
 - Simple instructions, each executes in one clock cycle – ***REALLY? Well, almost.***
 - Effective use of pipelining
 - Examples: ARM (snapdragon), MIPS (MIPS64), RISC-V (Raven-1)
- There are several others too:
 - VLIW (Very Large Instruction Word)
 - Minimal Instruction Set Computers (MISC)
 - Zero Instruction Set Computers (ZISC) etc.

COMPUTER ARCHITECTURE –INSTRUCTION SET ARCHITECTURE

27-Jan-2026

- Recap:
 - What is COA? Architecture vs Microarchitecture
 - Performance: Latency, Throughput and Speedup
 - Historical perspectives – Evolution and Basic Laws
- This/Next weeks Agenda (~next 4-5 lectures):
 - High level Computer Organization
 - Dive into Computer Architecture.
 - Instruction set Architecture (ISA)
 - (MIPS – case study) & Glance over other ISAs
- Logistics:
 - ~~Quiz 01 and Assignment 01 (posted)~~
 - Reading/reference on MIPS ISA is shared.



- Reading Materials:
 - Chapter 2 - COAHP
 - Chapters 3, 12 - COAWS

MIPS ISA (+ ASSEMBLY CODE)

- **MIPS** architecture: (**M**icroprocessor without **I**nterlocked **P**ipelined **S**tages)
 - AKA. **Load/Store Architecture**. Evolved from 32-bit ISA [**I**, **II..V**] to 64-bit ISA.
 - Developed by John Hennessy and colleagues at Stanford in the 1980's
 - Used in many commercial systems (Silicon Graphics, Nintendo, PS2, Cisco)
 - Development of **MIPS** architecture has largely been succeeded by **RISC-V**.
 - Recently, GlobalFoundries acquired MIPS (July 2025).
- Four Core underlying design principles of MIPS, as articulated by Hennessy and Patterson include:
 1. *Simplicity favours regularity*
 2. *Make the common case fast*
 3. *Smaller is faster*
 4. *Good design demands good compromises.*
- Once you've learned one architecture, it's easy to learn others!

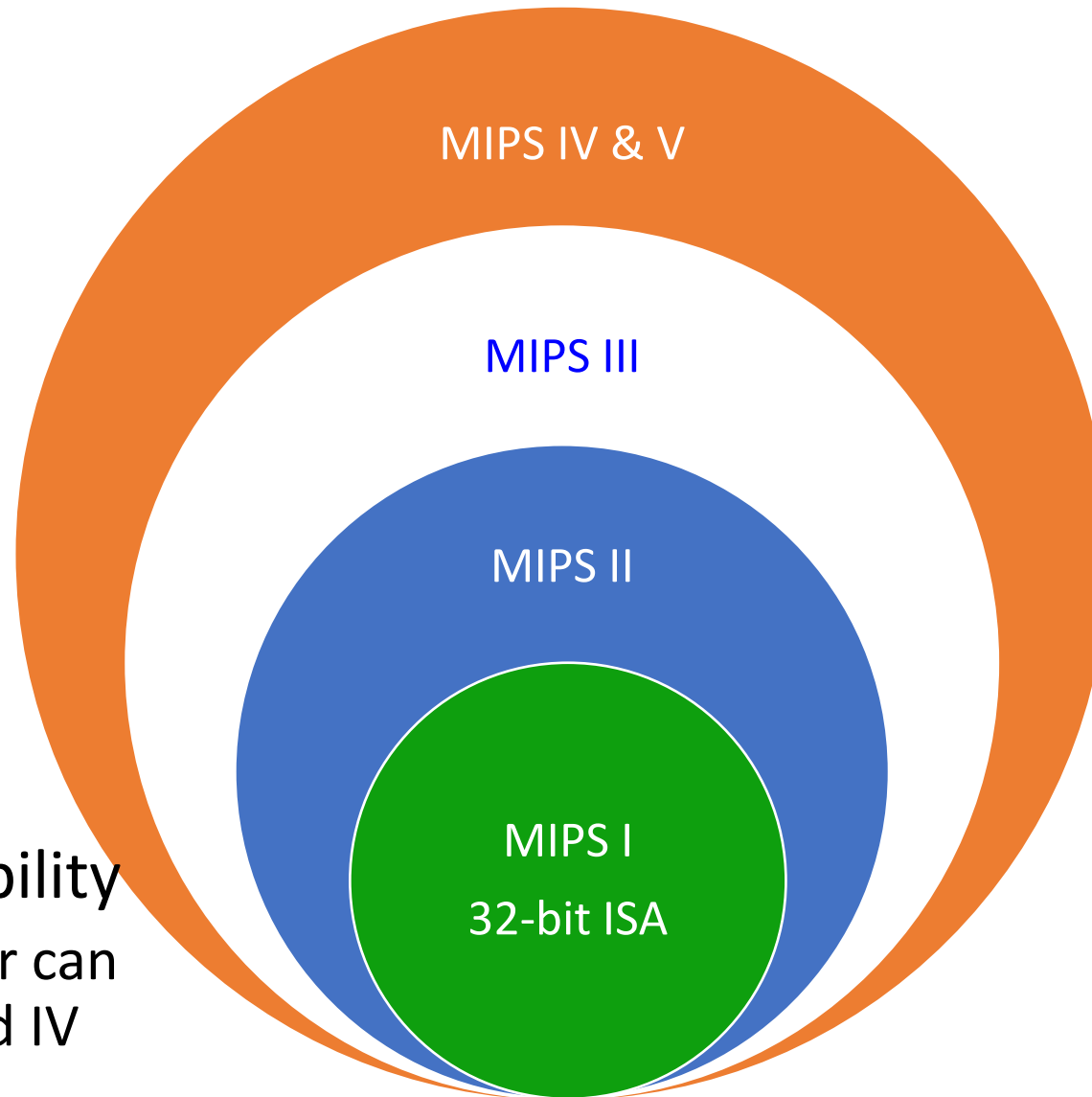


MIPS – THE ARCHITECTURE OF INCLUSIVE EVOLUTION!

- CPU Instructions are divided into following Functional Groups:

1. Load & Store
2. ALU
3. Jump & Branch
4. Miscellaneous
5. Co-processor

- Full Backward Compatibility
 - MIPS V based processor can execute MIPS I, II, III and IV based binary programs.



Register + Register
Addressing Mode for FPU

Unsigned word load
64-bit CPU transfer

Atomic Updates
64-bit coprocessor update

Baseline 32-bit ISA

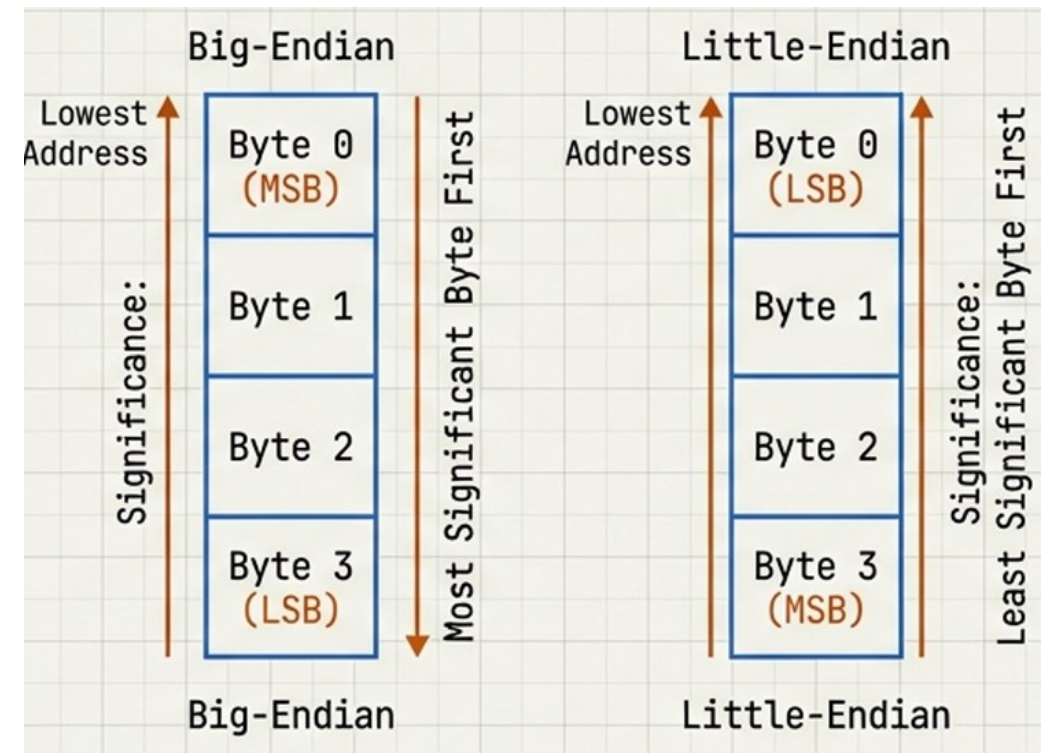
MIPS ISA IN A NUTSHELL

- Operands in MIPS ISA

Name	Example	Comments
32 registers	<code>\$s0-\$s7</code> , <code>\$t0-\$t9</code> , <code>\$zero</code> , <code>\$a0-\$a3</code> , <code>\$v0-\$v1</code> , <code>\$gp</code> , <code>\$fp</code> , <code>\$sp</code> , <code>\$ra</code> , <code>\$at</code>	Fast locations for data. In MIPS, data must be in registers to perform arithmetic, register <code>\$zero</code> always equals 0, and register <code>\$at</code> is reserved by the assembler to handle large constants.
2^{30} memory words	<code>Memory[0]</code> , <code>Memory[4]</code> , <code>...</code> , <code>Memory[4294967292]</code>	Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers.

- Data types Memory Addressing in MIPS

Type	Size
Byte	8-bit
Halfword	16-bit
Word	32-bit
Doubleword	64-bit



MIPS INSTRUCTIONS IN A NUTSHELL

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add \$s1,\$s2,\$s3	$\$s1 = \$s2 + \$s3$	Three register operands
	subtract	sub \$s1,\$s2,\$s3	$\$s1 = \$s2 - \$s3$	Three register operands
	add immediate	addi \$s1,\$s2,20	$\$s1 = \$s2 + 20$	Used to add constants
Data transfer	load word	lw \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Word from memory to register
	store word	sw \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Word from register to memory
	load half	lh \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Halfword memory to register
	load half unsigned	lhu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Halfword memory to register
	store half	sh \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Halfword register to memory
	load byte	lb \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Byte from memory to register
	load byte unsigned	lbu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Byte from memory to register
	store byte	sb \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Byte from register to memory
	load linked word	ll \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Load word as 1st half of atomic swap
	store condition. word	sc \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1; \$s1 = 0 \text{ or } 1$	Store word as 2nd half of atomic swap
	load upper immed.	lui \$s1,20	$\$s1 = 20 * 2^{16}$	Loads constant in upper 16 bits
Logical	and	and \$s1,\$s2,\$s3	$\$s1 = \$s2 \& \$s3$	Three reg. operands; bit-by-bit AND
	or	or \$s1,\$s2,\$s3	$\$s1 = \$s2 \$s3$	Three reg. operands; bit-by-bit OR
	nor	nor \$s1,\$s2,\$s3	$\$s1 = \sim (\$s2 \$s3)$	Three reg. operands; bit-by-bit NOR
	and immediate	andi \$s1,\$s2,20	$\$s1 = \$s2 \& 20$	Bit-by-bit AND reg with constant
	or immediate	ori \$s1,\$s2,20	$\$s1 = \$s2 20$	Bit-by-bit OR reg with constant
	shift left logical	sll \$s1,\$s2,10	$\$s1 = \$s2 \ll 10$	Shift left by constant
	shift right logical	srl \$s1,\$s2,10	$\$s1 = \$s2 \gg 10$	Shift right by constant

MIPS INSTRUCTIONS IN A NUTSHELL

Conditional branch	branch on equal	<code>beq \$s1,\$s2,25</code>	if ($\$s1 == \$s2$) go to $PC + 4 + 100$	Equal test; PC-relative branch
	branch on not equal	<code>bne \$s1,\$s2,25</code>	if ($\$s1 \neq \$s2$) go to $PC + 4 + 100$	Not equal test; PC-relative
	set on less than	<code>slt \$s1,\$s2,\$s3</code>	if ($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$	Compare less than; for beq, bne
	set on less than unsigned	<code>sltu \$s1,\$s2,\$s3</code>	if ($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$	Compare less than unsigned
	set less than immediate	<code>slti \$s1,\$s2,20</code>	if ($\$s2 < 20$) $\$s1 = 1$; else $\$s1 = 0$	Compare less than constant
	set less than immediate unsigned	<code>sltiu \$s1,\$s2,20</code>	if ($\$s2 < 20$) $\$s1 = 1$; else $\$s1 = 0$	Compare less than constant unsigned
Unconditional jump	jump	<code>j 2500</code>	go to 10000	Jump to target address
	jump register	<code>jr \$ra</code>	go to $\$ra$	For switch, procedure return
	jump and link	<code>jal 2500</code>	$\$ra = PC + 4$; go to 10000	For procedure call

MIPS INSTRUCTIONS: ADDITION

Instruction	Example	Meaning	Comments
add	add \$s1,\$s2,\$s3	$\$s1 = \$s2 + \$s3$	Three register operands

High-level code

```
a = b + c;
```

- **Operation:** Addition
- **Operands:** source operands (b, c), and destination operand a
 - b,c : read from
 - a: write to

MIPS assembly

```
add a, b, c
```

- **add:** mnemonic that indicates the operation to perform
- **b, c:** source operands on which the operation is performed (*augend and addend*)
- **a:** destination operand to which the result is written (*sum*)

MIPS INSTRUCTIONS: SUBTRACTION

subtract	sub \$s1,\$s2,\$s3	$\$s1 = \$s2 - \$s3$	Three register operands
----------	----------------------	----------------------	-------------------------

High-level code

```
a = b - c;
```

MIPS assembly

```
sub a, b, c
```

- Subtraction is similar to a addition, **only mnemonic changes**
- **sub:** mnemonic indicates what operation to perform
- **b, c:** source operands on which the operation is performed (*Minuend, Subtrahend*)
- **a:** destination operand to which the result is written (*Difference*)

Design Principle1: Simplicity favors regularity

- Consistent instruction format
- Same number of operands (two sources and one destination)
 - Makes it easier to encode and handle in hardware <We will learn the details soon...>

INSTRUCTIONS: MORE COMPLEX CODE

add	add \$s1,\$s2,\$s3	\$s1 = \$s2 + \$s3	Three register operands
subtract	sub \$s1,\$s2,\$s3	\$s1 = \$s2 - \$s3	Three register operands

High-level code

```
a = b + c - d;
```

MIPS assembly code

```
add t, b, c # t = b + c
sub a, t, d # a = t - d
```

More complex code is handled by multiple MIPS instructions.

```
a = b + c + d + e;
```

```
add t, b, c # t = b + c
add t, t, d # t = t + d
add a, t, e # a = t + e
```

• A single line of C code is converted into multiple lines of assembly code

• There can be multiple ways to execute the same HLI

```
add t, b, c # t = b + c
add s, d, e # t2 = d + e
add a, t, s # a = t + t2
```

- Some sequences are better than others...
- *the second sequence needs one **more temporary variables** (s), while it can perform first two instructions in **parallel**.*

Design Principle 2: Make the common case fast

- MIPS includes only simple, commonly used instructions.
- Hardware to decode and execute the instruction can be simple, small, and fast
- More complex instructions (that are less common) can be performed using multiple simple instructions. → *Increases Instruction count, but reduces CPI. ~~#PE~~*

```
a = b + c + d + e;
```

```
add a, b, c # a = b + c
add a, a, d # a = a + d
add a, a, e # a = a + e
```

INSTRUCTIONS: MORE COMPLEX CODE

High-level code

```
a = b + c - (d + e);
```

MIPS assembly code

```
add a, b, c # a = b + c  
sub a, a, d # a = a - d  
sub a, a, e # a = a - e
```

```
add a, b, c # a = b + c  
add t, d, e # t = d + e  
sub a, a, t # a = a - t
```

```
add t, d, e # t = d + e  
sub a, c, t # a = c - t  
add a, a, b # a = a + b
```


OPERANDS

- A computer needs a physical location from which to retrieve the operands

- A computer retrieves operands from:

- Registers
- Constants (also called immediate values)
- Memory

- Main Memory is slow – Registers for Faster Access.

- Most architectures have a small set of (fast) registers

- MIPS has *thirty-two* 32-bit registers

Name	Example	Comments
32 registers	<code>\$s0-\$s7, \$t0-\$t9, \$zero, \$a0-\$a3, \$v0-\$v1, \$gp, \$fp, \$sp, \$ra, \$at</code>	Fast locations for data. In MIPS, data must be in registers to perform arithmetic, register <code>\$zero</code> always equals 0, and register <code>\$at</code> is reserved by the assembler to handle large constants.

- MIPS is called a 32-bit architecture because it operates on 32-bit data

- A 64-bit version of MIPS also exists, but we will consider only the 32-bit version.

~300 cycles to hit Main Memory;
Signal needs time to travel

Registers & Caches:

Register (CPU Scratchpad): 1 cycle.

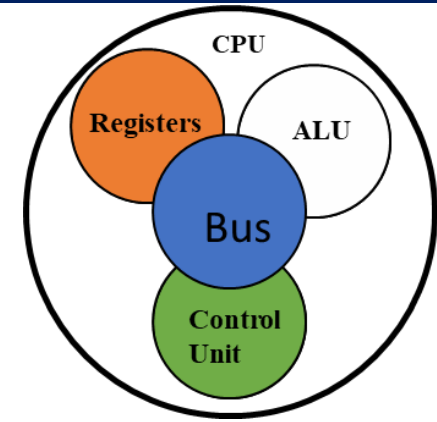
L1: 3-4 cycles, 32kB Inst/Data

L2: 8-14 cycles, 256kB (combined)

L3: 20-50, few MBs (combined)

MIPS OPERANDS: REGISTERS

- **Registers:** Fast, small (usually) storage locations inside the CPU.
 - Accessible within a single cycle access.
 - MIPS architecture defines 32 General-Purpose Registers (GPRs).
- **Usage Convention:** Written with a dollar sign (\$) before their name;
 - For example, register 0 is written “\$0”, pronounced “register zero” or “dollar zero”.
 - i) using numbers (\$0 - \$31) *or*
 - ii) using mnemonic names (\$zero, \$s0, \$t2, \$gp, etc..)
- **Certain registers are used for specific purposes:**
 - \$0 always holds the constant value 0;
 - \$1 (\$at) reserved for assembler; \$26,\$27 (\$K0,\$K1) – kernel registers reserved for OS (syscall).
 - the **saved registers (8)**, \$s0-\$s7, are used to hold variables.
 - the **temporary registers (10)**, \$t0 - \$t9, are used to hold intermediate values in larger computation.



0	1	2-3	4-7	8-15	16-23	24-25	26-27	28	29	30	31
\$zero	\$at	\$v0-\$v1	\$a0-\$a3	\$t0-\$t7	\$s0-\$s7	\$t8-\$t9	\$k0-\$k1	\$gp	\$sp	\$fp	\$ra
Constant 0	Assembler register	Result register	Argument register	Temporary registers	Saved registers	Temporary registers	Kernel registers	Global pointer	Stack pointer	Frame pointer	Return address
Preserved?	NA	NP (caller)	NP(caller)	NP(caller)	P(callee)	NP(caller)	NA	P(callee)	P(callee)	P(callee)	N/P

OPERAND: MIPS REGISTER

Design Principle 3: Smaller is Faster

- MIPS includes only a small number of registers. **WHY?**
- Just as retrieving data from a few books on your table is faster than sorting through 1000 books, retrieving data from 32 registers is faster than retrieving it from 1000 registers or a large memory. →
- **Incurs both space (cost) and time overheads.**

Additional Special Purpose Registers

\$PC	Program Counter: address of next instruction to be executed.
\$LO	Multiply: Stores the lower 32 bits; Divide: stores the quotient
\$HI	Multiply: Stores the upper 32 bits; Divide: stores the remainder

MIPS Register Set

Name	Register #	Usage	Saved
\$0	0	the constant value 0	NA (P)
\$at	1	assembler temporary	NA (NP)
\$v0-\$v1	2-3	procedure return values	Caller (NP)
\$a0-\$a3	4-7	procedure arguments	Caller (NP)
\$t0-\$t7	8-15	temporaries	Caller (NP)
\$s0-\$s7	16-23	saved variables	Callee(P)
\$t8-\$t9	24-25	more temporaries	Caller (NP)
\$k0-\$k1	26-27	OS temporaries	NA (NP)
\$gp	28	global pointer	Callee(P)
\$sp	29	stack pointer	Callee(P)
\$fp	30	frame pointer	Callee(P)
\$ra	31	procedure return address	Caller (NP)

Callee Saved are also known as Preserved (P) Registers.
Caller Saved are the Non Preserved (NP) Registers.

II. MIPS OPERANDS: CONSTANTS/IMMEDIATE

High-level code

```
a = b + c;
```

- Source and destination operands are now in registers.

```
a = a + 4;  
b = a - 12;
```

MIPS assembly

```
add a, b, c  
# $s0 = a  
# $s1 = b  
# $s2 = c  
add $s0, $s1, $s2
```

```
# $s0=a, $s1=b  
addi $s0, $s0, 4  
addi $s1, $s0, -12
```

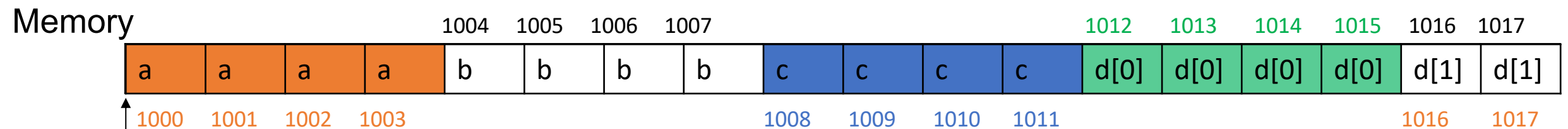
- ***addi*** illustrates the use of constants or immediates
 - Immediates don't require a register or memory access
 - Called immediates because they are directly available (encoded bits in the instruction)
- **add immediate instruction adds an immediate to a variable (held in a register)**
 - An immediate is a 16-bit two's complement number.
- **Is subtract immediate (subi) necessary?**

III. MIPS OPERANDS: MEMORY

- Too much data to fit in only 32 registers → Store more data in memory.
 - Memory is large but slow (~1 cycle to access register vs. ~300 cycles for mem)
- The compiler organizes data in memory... it knows the location of every variable (saved in a table)... it can fill in the appropriate mem-address for load-store Instructions.
 - Keep commonly used variables kept in registers
- Combination of registers and memory, helps access a large amount of data fairly quickly.

How is memory-address determined? (assuming `sizeof(int) = 4`)

`int a, b, c, d[10];`



Base address = 1000

BIG-ENDIAN AND LITTLE-ENDIAN MEMORY

- How to number bytes within a word?
- Word address is the same for big- or little-endian
 - Little-endian:** byte numbers start at the little (least significant) end
 - Big-endian:** byte numbers start at the big (most significant) end
- From Jonathan Swift's *Gulliver's Travels*: where the Little-Endians broke their eggs on the little end of the egg and the Big-Endians broke their eggs on the big end.
 - As indicated by the farcical name, it doesn't really matter which addressing type is used – except when the two systems need to share data!

Big-Endian

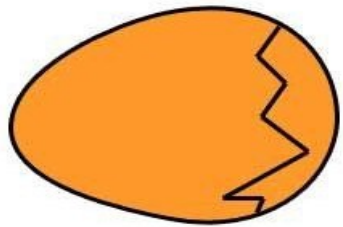
	Byte Address			
	⋮			
d =	C	D	E	F
c =	8	9	A	B
b =	4	5	6	7
a =	0	1	2	3

MSB LSB

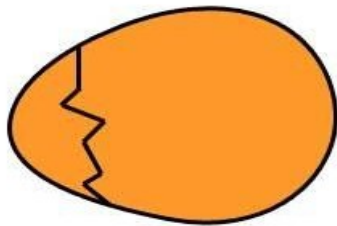
Little-Endian

	Byte Address			
	⋮			
	F	E	D	C
	B	A	9	8
	7	6	5	4
	3	2	1	0

MSB LSB



BIG ENDIAN - The way people always broke their eggs in the Lilliput land



LITTLE ENDIAN - The way the king then ordered the people to break their eggs

```
int a = 0xBADAADDA
int b = 0xF00DCAFE
int c = 0xBABEFACE
int d = 0xDEADC0DE
str s = "DEADCODE"
```

Big-Endian

	Byte Address			
	⋮			
d =	DE	AD	C0	DE
c =	BA	BE	FA	CE
b =	F0	0D	CA	FE
a =	BA	DA	AD	DA

MSB LSB

Little-Endian

	Byte Address			
	⋮			
d =	DE	C0	AD	DE
c =	CE	FA	BE	BA
b =	FE	CA	0D	F0
a =	DA	AD	DA	BA

MSB LSB

BIG-ENDIAN AND LITTLE-ENDIAN EXAMPLE

- Suppose \$t0 initially contains 0x23456789. After the following program is run on a big-endian system, what value does \$s0 contain? In a little-endian system?

```
sw $t0, 0($0)
lb $s0, 1($0)
```

Big-endian:

0x00000045

Little-endian:

0x00000067

Big-Endian

Byte Address	0	1	2	3
Data Value	23	45	67	89
	MSB		LSB	

Word
Address
0

Little-Endian

3	2	1	0	Byte Address
23	45	67	89	Data Value
MSB		LSB		

MIPS DATA TRANSFER INSTRUCTIONS

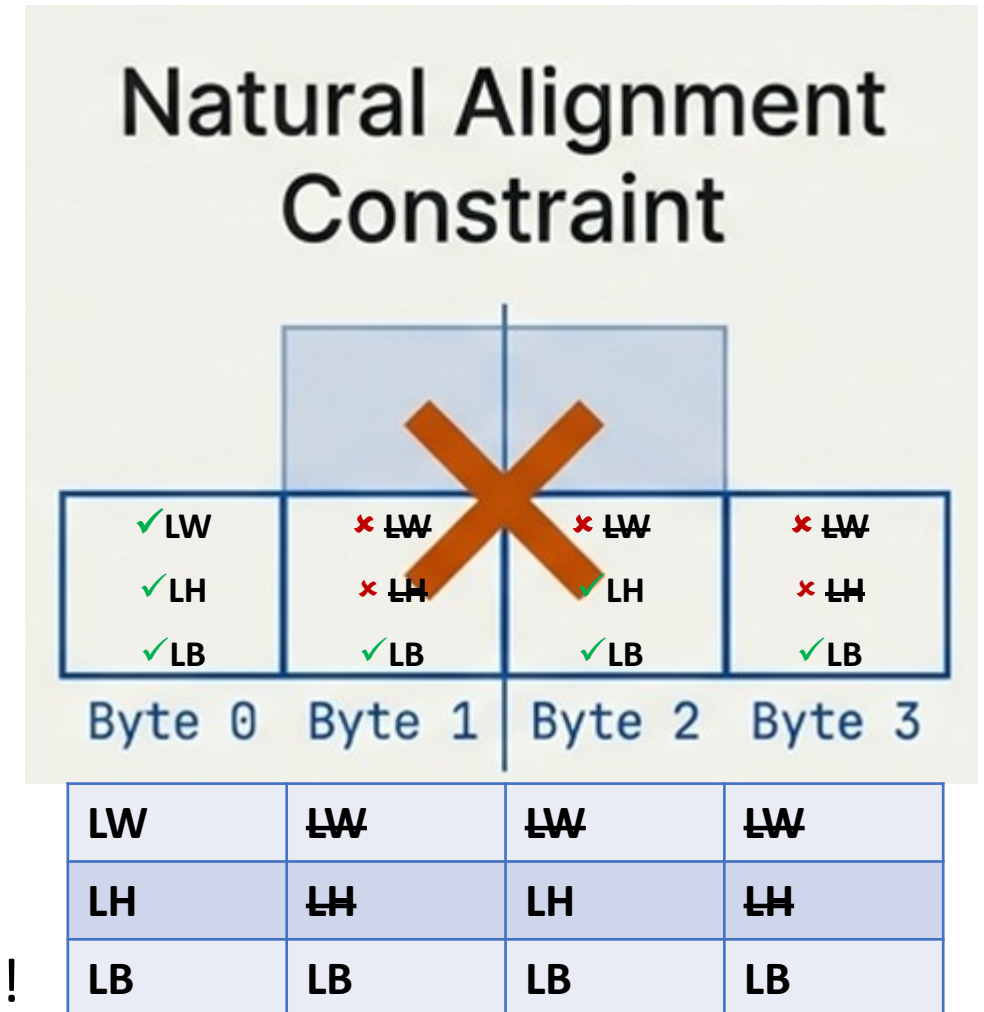
Category	Instruction	Example	Meaning	Comments
Data transfer	load word	lw \$s1, 20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Word from memory to register
	store word	sw \$s1, 20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Word from register to memory
	load half	lh \$s1, 20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Halfword memory to register
	load half unsigned	lhu \$s1, 20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Halfword memory to register
	store half	sh \$s1, 20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Halfword register to memory
	load byte	lb \$s1, 20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Byte from memory to register
	load byte unsigned	lbu \$s1, 20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Byte from memory to register
	store byte	sb \$s1, 20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Byte from register to memory
	load linked word	ll \$s1, 20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Load word as 1st half of atomic swap
	store condition. word	sc \$s1, 20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1; \$s1 = 0 \text{ or } 1$	Store word as 2nd half of atomic swap
	load upper immed.	lui \$s1, 20	$\$s1 = 20 * 2^{16}$	Loads constant in upper 16 bits

- Load Instructions fetch data from Memory in to the Register.
- Store Instructions save data to Memory from the Register.

LOAD AND STORE VARIANTS

- Registers are 32 bits, but operations can be on 8/16/32 bit signed and/or unsigned data.
- Hence, we need instructions to load and store 8 and 16-bit data as well.

Size	Load (Signed)	Load (Unsigned)	Store
Byte (8-bit)	LB	LBU	SB
Halfword (16-bit)	LH	LHU	SH
Word (32-bit)	LW	LWU (MIPS III)	SW



- Both Load & Store need to clearly specify the data size.
- Store does not need a unsigned version, but Load does!
- Alignment constraint must be honoured for Load & Store

ACCESSING OPERANDS FROM MEMORY

Convert to assembly: Assume the data (a, b,c,d[]) is stored starting at base address 1000 contiguously.

```
addi $gp, $zero, 1000 # place base address in $gp; # $zero is a register that always equals 0.
```

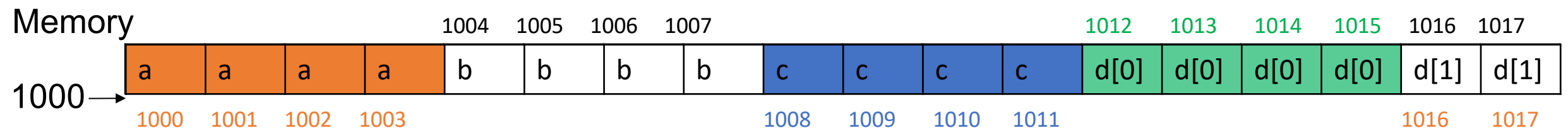
```
lw $s0, 0($gp)      # brings value of a into register $s0
lw $s1, 4($gp)      # brings value of b into register $s1
lw $s2, 8($gp)      # brings value of c into register $s2
lw $s3, 12($gp)     # brings value of d[0] into register $s3
lw $s4, 16($gp)     # brings value of d[1] into register $s4
lw $s5, 20($gp)     # brings value of d[2] into register $s5
```

High-level code

```
int a, b, c, d[10];
d[3] = d[2] + a;
```

MIPS assembly

```
#Assembly (same assumptions as previous example):
lw $s0, 0($gp) # a is brought into $s0
lw $s1, 20($gp) # d[2] is brought into $s1
add $t1, $s0, $s1 # the sum is stored in $t1
sw $t1, 24($gp) # $t1 is stored into d[3]
```



SIMPLE SWAP FUNCTION AS AN EXAMPLE

- Consider the Swap function as below:

High-level code

```
void swap(int v[], int k);  
{  
    int temp;  
    temp = v[k]  
    v[k] = v[k+1];  
    v[k+1] = temp;  
}
```

1. Map **temp** variable to some register
2. Compute address of **V[k]** and **V[k+1]**
3. **Load V[k]** and **V[k+1]** to reg. **\$t0** & **\$t1**.
4. **Swap data on registers \$t0 & \$t1?**
5. **Store swapped data to V[k] and V[k+1]**

MIPS assembly code

```
swap:
```

Assume k (index) is in reg \$S1 and base address of V is in \$S2.

MIPS ADDRESSING MODES (REGISTER, IMMEDIATE, MEMORY)

- MIPS uses five different addressing modes to determine the location of data or instruction operands.

Example MIPS code

```
#Register Direct (R)
add $s0, $s1, $s2
jr $s0    #ETA=[$s0]
jalr $s0, #ETA=[$s0]
```

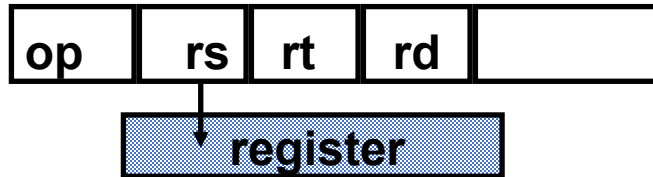
```
# Immediate (I)
addi $s0, $s1, 4
addi $s1, $s0, -12
```

```
# Base+Index (I)
lw $s0, 16($s2)
```

```
# PC Relative (I)
beq $s0, $s3, 2500
```

```
# Pseudo direct (J)
J 2500
jal 2500
```

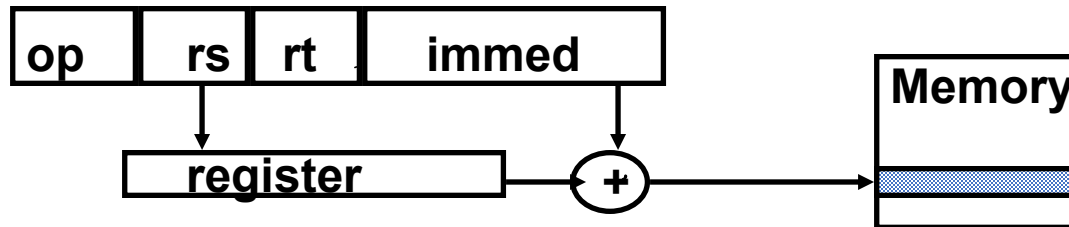
Register (direct)



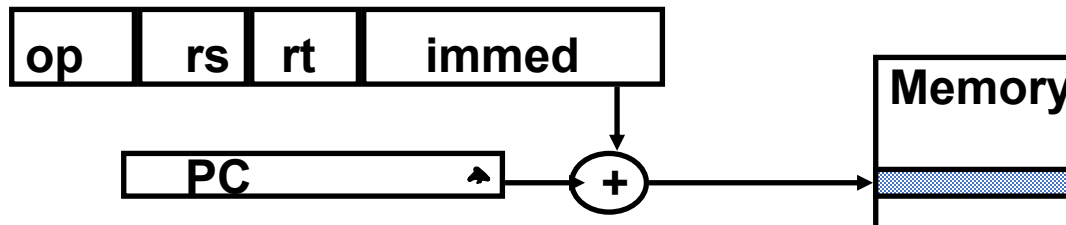
Immediate



Base+index
(Displacement)



PC-relative



$$ETA = ((PC+4) + (immed \ll 2))$$

Pseudo-direct

$$ETA = ((PC+4) \& 0xF0000000) | (immed \ll 2)$$

MIPS ADDRESSING MODES (REGISTER, IMMEDIATE, MEMORY)

- MIPS uses five different addressing modes to determine the location of data or instruction operands.

Addressing Mode	Format	Effective Address (EA) / Operand	Value Extension	Instruction Examples
Register	R	Operand = Reg[<i>rs</i>]	N/A	add, sub, and, or, slt
Immediate	I	Operand = 16-bit field	Signed or Zero-Ext	addi, ori, andi, slti
Base	I	$EA = \text{Reg}[\textit{base}] + 16\text{-bit Offset}$	Signed (2's Comp)	lw, sw, lb, sb, lh, sh
PC-Relative	I	$EA = (PC + 4) + (16\text{-bit Offset} \times 4)$	Signed (2's Comp)	beq, bne, blez, bgtz
Pseudo-Direct	J	$EA = (PC + 4)[31 : 28] \mid (\text{Index} \times 4)$	Unsigned	j, jal